

The agent improvement loop starts with a trace

LangChain · Sam Crowder · 2026-03-31 · 原文 / Source

Key Takeaways

- An agent is a system around a model with several layers you can update: the model weights, the orchestration code, and the context (prompts, instructions, skills). Knowing what to change requires evidence from traces.
- Traces can come from anywhere: staging, test runs, benchmarks, local development, and especially from production. The improvement loop is the same regardless of source.
- The loop requires enriching traces with evals and human feedback, identifying failure patterns, making targeted changes, and validating before shipping. Each cycle generates better data and more reliable iteration.
- LangSmith connects every step of this loop, from the first trace to the CI/CD gate that prevents regressions from shipping.

The agent improvement loop is simple in principle: get traces, enrich them, improve from them, repeat. This guide walks through how to do that in practice.

Improving an agent systematically requires a feedback loop. You collect traces of agent behavior, enrich them with evaluations and human feedback, identify what's failing and why, make targeted changes, and validate that those changes worked before shipping. Then you repeat from a higher baseline.

This loop is powered by traces. These traces can come from many places – from staging environments, benchmark runs, local development, and especially from production. What matters is collecting these traces, enriching them, and using that data to improve the system.

This guide walks through each step of that feedback loop.

Traces are the raw material

Harrison put it directly: "In software, the code documents the app; in AI, the traces do."

In a traditional application, the code is the authoritative record of what the system does. You can read it, reason about it, test against it, and understand every behavior in principle.

In an agentic system, the code tells you what the agent is *allowed* to do. The traces tell you what it actually *did* in this run, with this input, under these conditions.

TRACE

☰

Waterfall

⚙️

↶

ChatOpenAI

Tools

Company

Input

SYSTEM

You are a LangChain expert and a world-class researcher, here to assist with any and all questions or issues with LangChain, LangGraph, LangSmith, or any related functionality. Users may come to you with questions or issues. Based on the conversation below, generate a plan for how you will research the answer to their question.

The plan should generally not be more than 3 steps long, it can be as short as one. The length of the plan depends on the question.

You have access to the following documentation sources:

- Conceptual docs
- Integration docs
- How-to guides

You do not need to specify where you want to research for all steps of the plan, but it's sometimes helpful.

Output

HUMAN

show me a langgraph workflow that uses runtime config for prompt

RetrievalGraph ✓

12.00

create_research_plan 1.38s

RunnableSequence 15.76s

ChatOpenAI gpt-5-mini 1.36s

JsonOutputKeyTo... 0.00s

conduct_research 2.48s

ResearcherGraph 2.48s

generate_queries 0.88s

RunnableSequence 15.76s

ChatOpenAI gpt-4.1-mini

JsonOutputKeyTo... 0.00s

retrieve_in_parallel 0.00s

retrieve_documents 1.57s

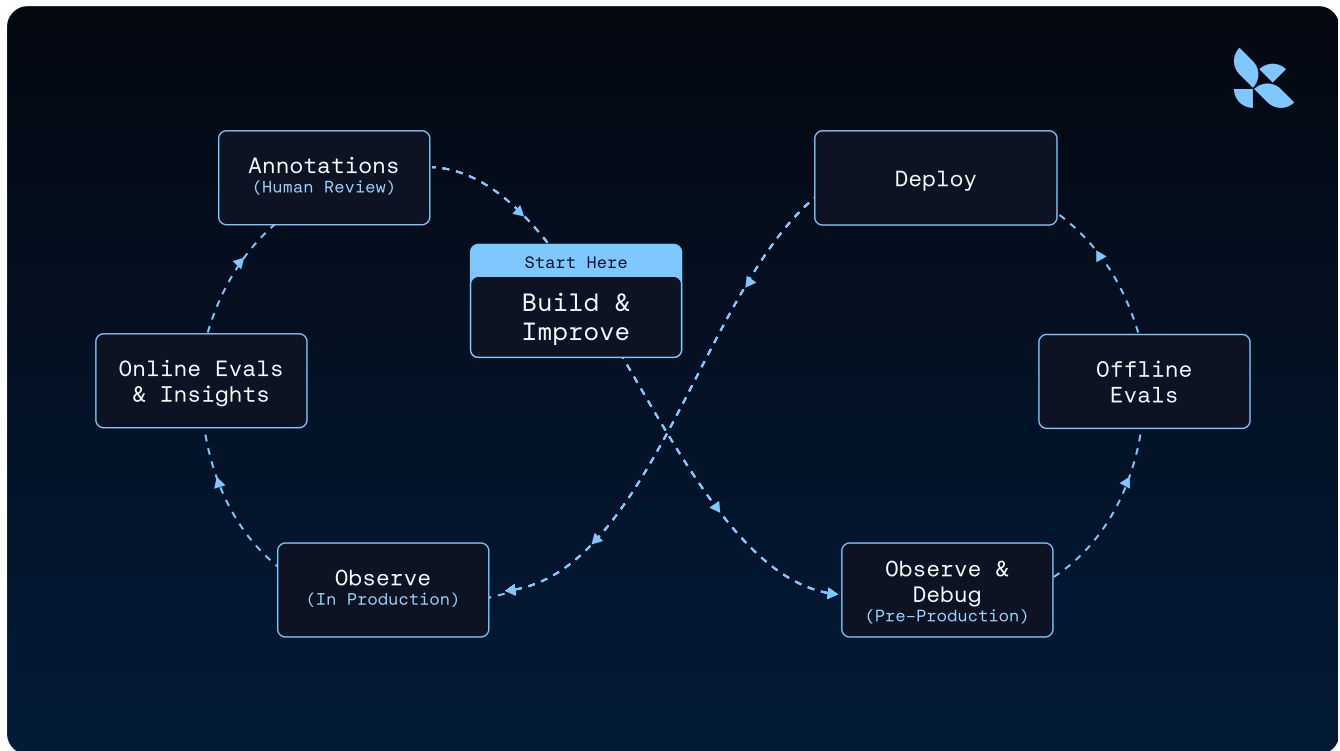
VectorStoreRetriever 1.39s

retrieve_documents 1.21s

A trace captures the full execution of an agent run: every LLM call, every tool invocation, every retrieval step, every intermediate output, and the sequence of decisions connecting them. It's the record of what the agent actually did with this input, under these conditions, in this run.

A raw trace tells you what happened. An enriched trace, scored by evaluators and annotated by reviewers, tells you what to do about it. The rest of this loop is about building that enrichment layer and using it to make targeted, validated improvements.

The agent improvement loop



Once you have traces, improving an agent becomes concrete and repeatable. The loop looks like this:

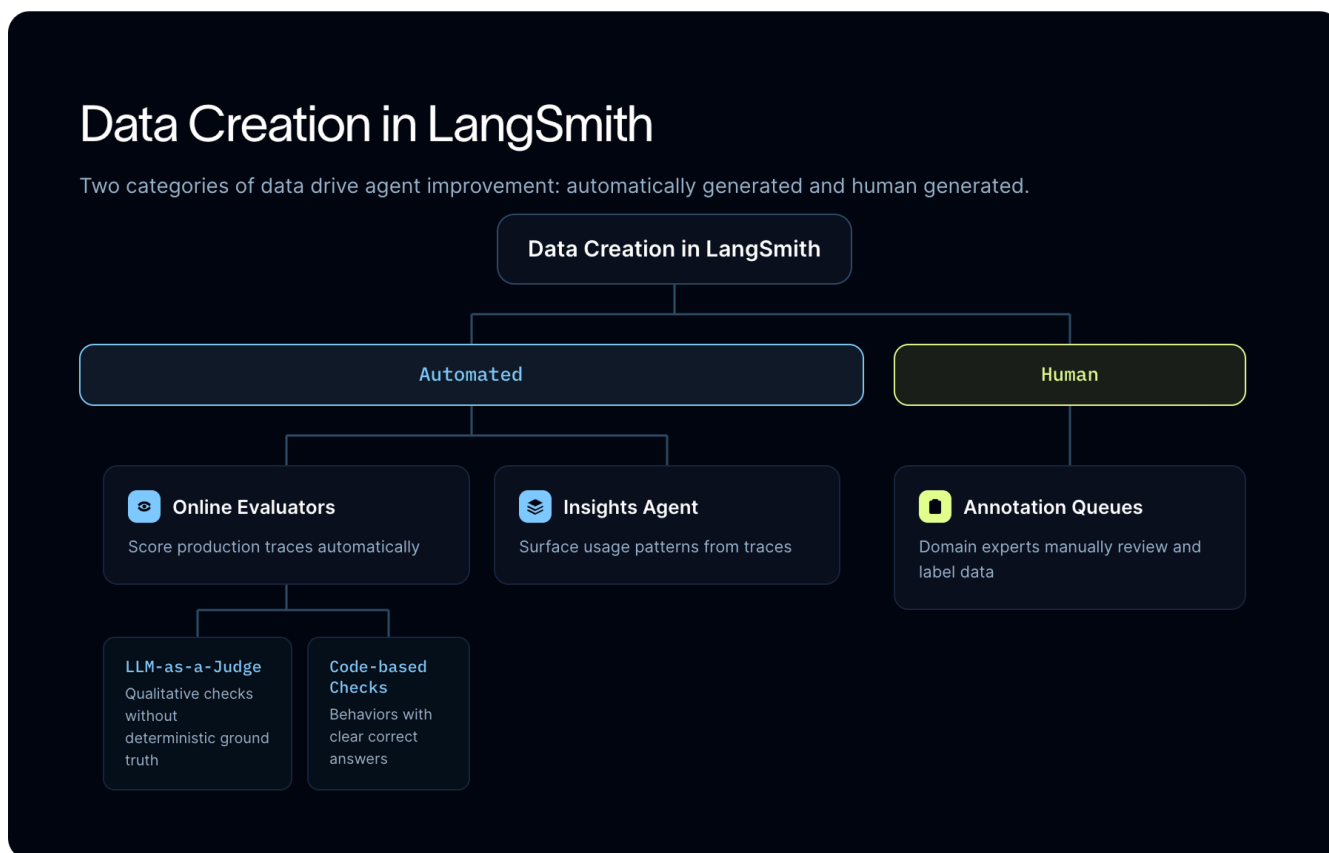
- 1. Build and improve:** Start with what you know – an agent, a task, and a hypothesis about what could be better. Developers review traces with negative scores, filter for failure patterns, and inspect the trajectory that produced a bad outcome. Instead of guessing at what to fix, they work backward from observed behavior. The failure modes that emerge from real traces become the input to code and prompt changes.
- 2. Observe and debug (pre-production):** Developers run the updated agent in a staging environment. Traces reveal whether the fix behaves as intended before any formal evaluation.
- 3. Offline evals:** Enriched traces get converted into reproducible test cases. A recurring failure mode becomes an evaluator. A set of real inputs that exposed a problem becomes a dataset. Before shipping, developers run the offline eval suite against the updated agent, producing a concrete before-and-after comparison. If the fix works, scores improve. If it introduces a regression, that surfaces before it reaches users. Passing evaluations get added to the permanent test suite.
- 4. Deploy:** The fix ships, and new traces start accumulating. The next cycle begins from a higher starting point.
- 5. Observe (in production):** Every agent run in production generates a trace: inputs, outputs, trajectory, tool calls, token usage, latency. This is the raw material for the next cycle, and the source of truth for what the agent actually did.
- 6. Online evals and Insights:** Raw traces become more useful when they carry additional signals. Automated evaluators score outputs continuously. Insights reports surface usage patterns, failure modes, and edge cases across large volumes of traces.

7. Annotations Human reviewers annotate selected traces with ratings, corrections, and comments. Each enrichment layer adds context to the raw behavioral record, building the labeled data that feeds back into the next build cycle.

The loop compounds because each cycle generates better data. More traces mean more examples of failure modes. More examples mean more precise evaluations. More precise evaluations mean more reliable iteration.

How LangSmith generates data from traces automatically

There are two categories of data that drive agent improvement: automatically generated and human generated. LangSmith helps you create both. To generate data automatically, you can use online evaluators and Insights Agent.



Online evaluators

Online evaluators run automatically on production traces, scoring outputs against configurable quality criteria. You can configure them to run on all traces, a sampled subset, or filtered subsets based on specific criteria.

The grading method depends on what you're evaluating.

For qualitative dimensions without deterministic ground truth, helpfulness, tone, relevance, policy adherence, factual plausibility, use an LLM-as-a-judge. The evaluator calls an LLM to assess not just the final response but the full trajectory: did the agent use the right tools, in the right order, with the right parameters?

For behaviors with clear right answers, use code-based checks. Schema validation, exact-match conditions, format conformity, business rule compliance, and tool correctness can all be evaluated deterministically, and doing so is faster and cheaper than routing them through an LLM judge.

Recurring insights and reports

< coding-agent-insights

> Category	Description	Runs	Effectiveness	hallucination	user
> Dev Experience	Tutorials, examples, debugging, prompts, and structured-output patterns for developer productivity.	126	6.53	0.73	1.00
> Platform	Deployment, self-hosting, API keys, tokens, infrastructure configuration, and system architecture.	79	6.63	0.29	1.00
> Orchestration	Graph routing, async runtimes, streaming, multi-agent flows, and system patterns.	53	4.3	0.29	
> Indexing	Embeddings, vector stores, similarity search, retrieval systems, and citation strategies.	37	8.00	0.63	
> Frontend	UI frameworks, Streamlit/React issues, UX, state management, and application integration.	18	8.30	0.60	
> Security	Compliance, data protection, SOC2, GDPR, licensing, and regional hosting controls.	15	8.88	0.50	
> Evaluation	Automated evaluation, tracing, metrics, result sharing, and model performance assessment.	12	6.71	0.25	

LangSmith's Insights Agent runs automated clustering over production traces to surface usage patterns, failure modes, and edge cases. This is different from monitoring: you're not tracking metrics you already defined, you're discovering patterns you didn't know to look for.

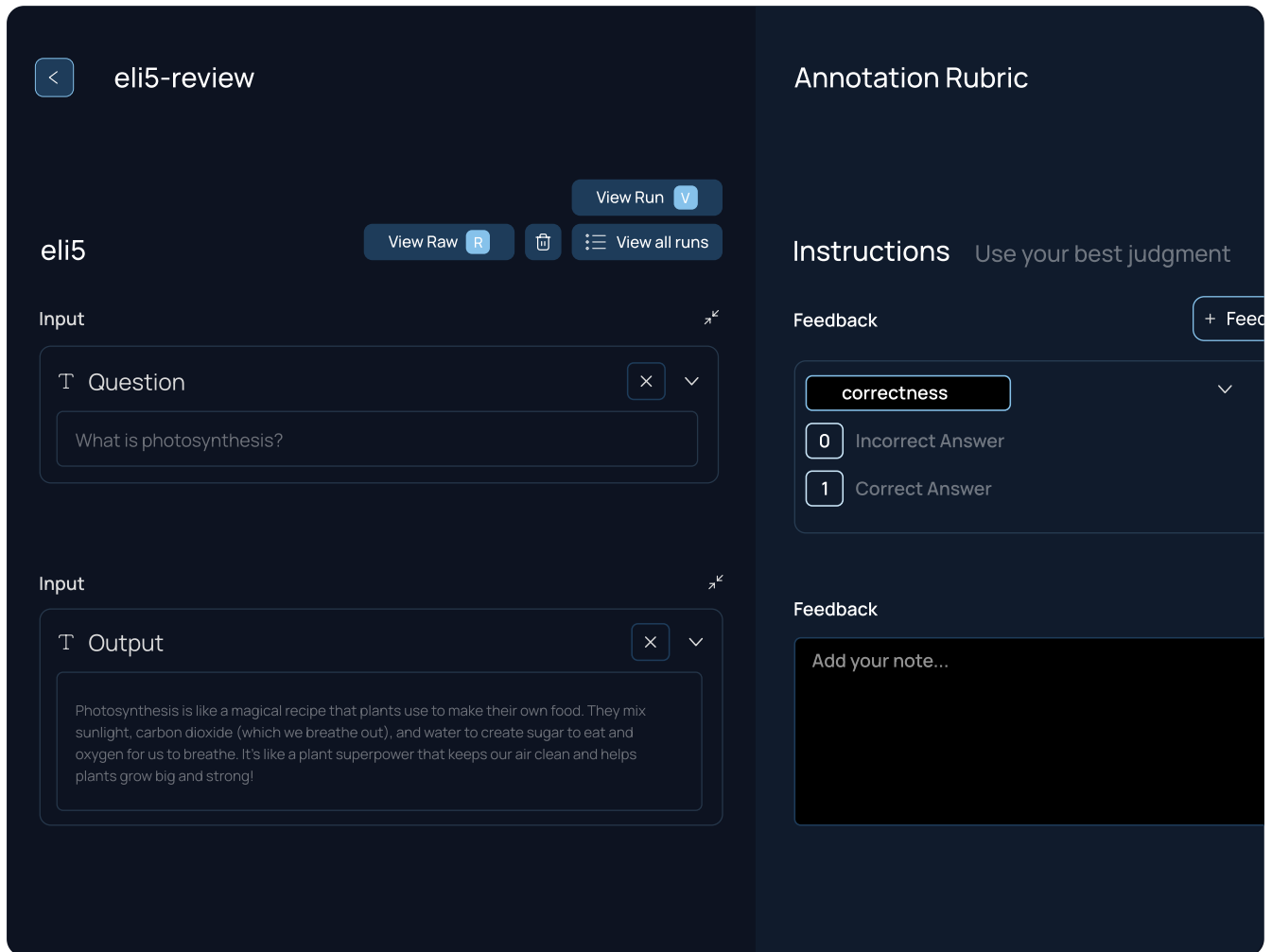
A team managing a customer-facing agent might ask: "What are users actually trying to do with this agent?" Insights Agent can analyze thousands of traces, group them by intent, and surface the top categories, including ones no one anticipated. The same analysis applied to traces with negative feedback or low scores reveals where the agent is consistently falling short and why.

Where automation stops: human judgment still matters

Automated evaluators and insights scale well, but they don't replace human judgment.

Some agent behaviors can only be assessed by someone with domain expertise. A legal research agent that cites plausible-sounding but inaccurate precedents might fool an LLM judge. A medical information agent that gives technically correct but clinically inappropriate guidance looks fine to an automated check. Nuanced failures in specialized domains require reviewers who understand what "correct" actually means.

That's where annotation queues come in.



Teams can route selected production traces into annotation queues using filters: traces with low automated scores, traces from a specific feature area, traces that received thumbs-down feedback from end users. Reviewers see the full context and can leave ratings, corrections, comments, and edited outputs.

There are four main ways teams use annotation queues in practice:

- **Aligning online evaluators:** Reviewers label traces to calibrate LLM-as-a-judges. When reviewers and the automated evaluator disagree, those labeled examples help tune the grader until its scores reflect human judgement.
- **Creating ground truth for offline datasets:** Reviewers label the correct final output for a trace. These become the expected answers in your offline eval suite, letting you test future versions for correctness against production inputs.
- **Scoring open-ended outputs:** When there's no single correct answer, reviewers label the criteria that define a good response. This structured feedback becomes the basis for evaluators on dimensions that are too nuanced for exact-match checks.
- **Natural language annotations:** Reviewers attach freeform comments and corrections to traces. These flow into Insights Agent analysis, surfacing patterns that scores alone won't show.

It's worth distinguishing between the first use case and the rest. Annotating to align online evaluators improves your live monitoring: you're making continuous, automated scoring more accurate. The other three are primarily about building offline datasets, creating the ground truth and quality labels that let you test a fix before it ships.

Two reviewer profiles are common in practice.

- **General reviewers:** Contractors, annotators, customer success teams can assess surface-level quality signals. They judge if the response was helpful, if it was accurate relative to visible information, and if the tone was appropriate.
- **Domain experts:** Product managers, SMEs, and specialists in the field the agent serves can judge whether the agent behaved correctly in context, including failures that automation will miss entirely.

Often, at this point in time, you still need a human-in-the-loop.

What to do with enriched traces: build and improve

Enriched traces become the raw material for understanding where an agent consistently fails.

The pattern that emerges across multiple traces is more actionable than any individual example. You'll start to see that your agent consistently misunderstands queries of a certain type or always selects the wrong tool in a particular context.

Pattern-level understanding is hard to gain by spot-checking individual runs. It requires data at scale, with consistent labels, from real production behavior.

The fix depends on what the traces reveal. If the agent is selecting the wrong tool for a class of queries, that might mean updating tool descriptions or adding routing logic. If the reasoning drifts partway through a multi-step task, a more constrained system prompt or breaking the task into smaller, more focused steps. If the agent produces outputs that are factually correct but miss the user's actual intent, that's usually a prompt-level issue and requires clarifying what "good" looks like in the instructions. And sometimes the trace reveals a structural problem: the agent needs a different tool entirely, or the workflow needs a human-in-the-loop checkpoint at a specific decision point.

Each of these changes is informed by specific, observed behavior rather than hypothetical failure modes. A developer is rewriting a prompt because they can see exactly which traces failed, how they failed, and what the annotated feedback says about why.

Offline evaluations then make these prompt and code changes measurable.

Turning production failures into offline evaluations

Once you've identified what to fix, you need a way to test that the fix actually works. That's where offline evaluations come in.

The dataset for those evals should come from production: real traces, real queries, real failures.

What those evaluations actually measure depends on what the annotation work produced. There are two distinct approaches:

- **Ground truth correctness:** When reviewers have labeled the correct final output for a trace, you can test directly for correctness. Run your refined agent against the dataset and compare the agent's output to the labeled ground truth. If the fix works, scores improve. If it introduces a regression, the eval catches it before it reaches users.
- **Criteria-based scoring:** Not every output has a single correct answer. For open-ended tasks, reviewers label the criteria that define a good response rather than the response itself. Offline evals use those criteria to score outputs from updated versions, letting you measure improvement on dimensions like relevance, completeness, or tone without requiring an exact match.

Every failure mode you encode as an eval should stay in your test suite permanently. That creates a durable record of what your agent has learned to handle, and a gate that ensures future changes don't reintroduce problems you already solved.

Online + offline evals together

Online + Offline Evals Together

Two evaluation modes create a bridge between production observation and safe iteration — turning agent improvement into something measurable.

	 ONLINE EVALS	 OFFLINE EVALS
Environment	Production — monitors live agent behavior continuously	Development — controlled experiments on curated datasets
Purpose	Catch quality drift, surface emerging failure patterns, flag traces for human review	Validate a change before it ships — confirm whether a fix actually addresses the problem
Timing	Continuous — always running against live traffic	Pre-deployment — run before any change reaches production
Key question	"What's going wrong?"	"Does the fix actually work?"
Output	Alerts, flagged traces, quality scores over time	Pass/fail results, comparative scores across versions
Applies to	All live interactions — samples or full coverage	Prompt changes, model updates, workflow modifications, architecture changes

Online evaluators monitor live behavior continuously. They catch quality drift, surface emerging failure patterns, and flag traces for human review. But they don't let you validate a change before it ships.

Offline evaluations help with that. They're controlled experiments you run on curated datasets in development, before any change reaches production.

Together, they create a bridge between production observation and safe iteration. Online evals tell you what's going wrong. Offline evals confirm whether your fix actually addresses it.

Every failure mode you encode as an eval should stay in your test suite permanently. That creates a durable record of what your agent has learned to handle, and a gate that ensures future changes don't reintroduce problems you already solved.

Every prompt change, model update, workflow modification, or architecture change should run against the accumulated eval suite before shipping. Running the same eval sets continuously and comparing scores across versions and model configurations turns the improvement loop into something measurable. You can demonstrate that each iteration produced a better agent, not just a different one.

Coding agents in the loop

The improvement loop is becoming more automated, and tracing stays at the center of that too..

The LangSmith CLI and Skills give coding agents expert-level access to LangSmith data directly from the terminal. When equipped with LangSmith Skills, on our eval set, Claude Code's performance jumped from 17% to 92%.

In practice, a developer can instruct a coding agent to pull the last 30 days of production traces, isolate traces with thumbs-down feedback, identify the failure patterns they represent, draft evaluations from those examples, and propose prompt or code changes to address them. All of that happens within a single terminal session, grounded in real behavioral data.

But a coding agent without trace data makes changes based on incomplete information. It will propose fixes that look reasonable from a code-review perspective but miss the actual failure mode because they can't see the execution trajectory that produced it. A coding agent working from enriched traces is working from the same information a senior engineer would use.

Tracing is the cornerstone of your agent improvement loop

Reliable agents aren't built from debugging individual traces. They're built from a trace-centered improvement loop.

The loop begins with tracing and returns to tracing. Every evaluator runs on traces. Every annotation is attached to a trace. Every offline dataset is built from traces. Every regression test validates against what was observed in real traces. The coding agent that proposes the next fix reads from traces to do it.

That is why tracing is not just a debugging tool. It's the primitive that makes the entire improvement loop possible, the foundation from which all evaluation, all human feedback, and all systematic improvement is derived.

The loop starts with a trace. And the next loop starts with the trace that comes back.

Additional reading

- ["Agent observability powers agent evaluation"](#)
- ["You don't know what your agent will do until it's in production"](#)
- [LangSmith Docs - "Offline evaluation types"](#)
- [LangSmith Docs - "Online evaluation types"](#)
- [LangSmith Docs - "Annotation queues"](#)
- [LangSmith Docs - "Insights"](#)